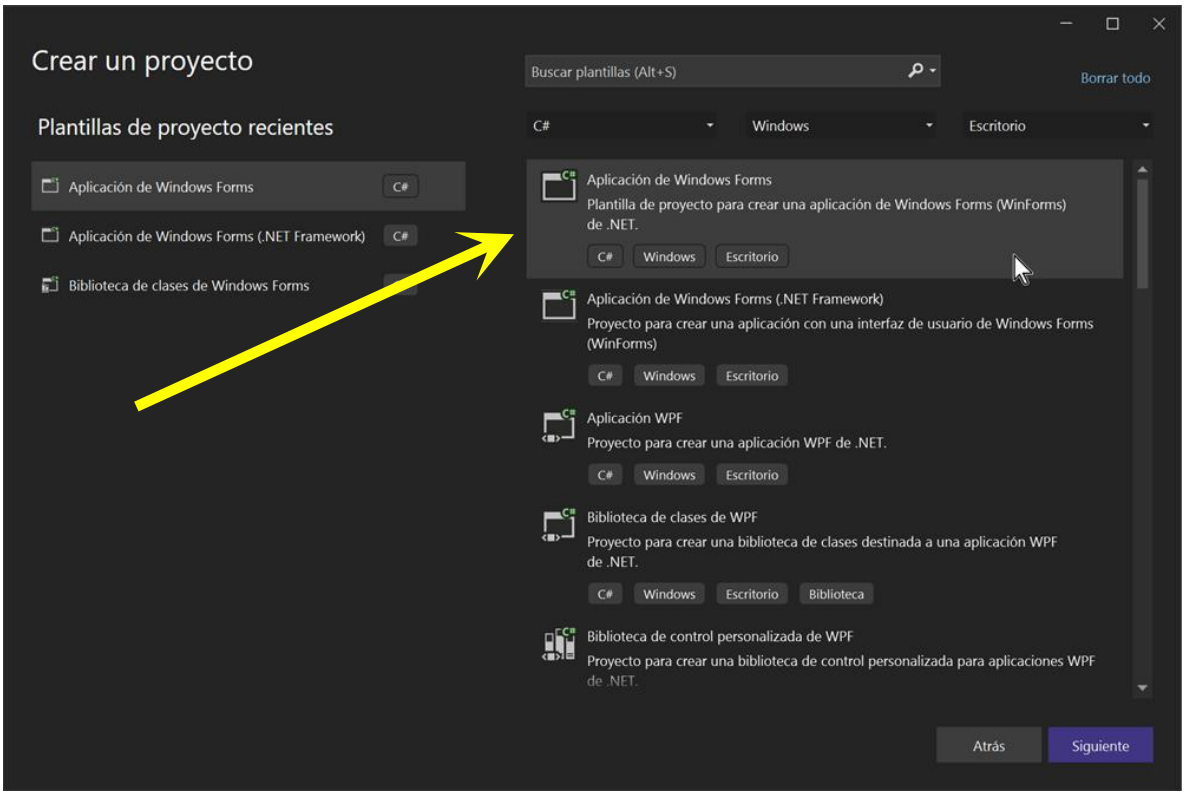
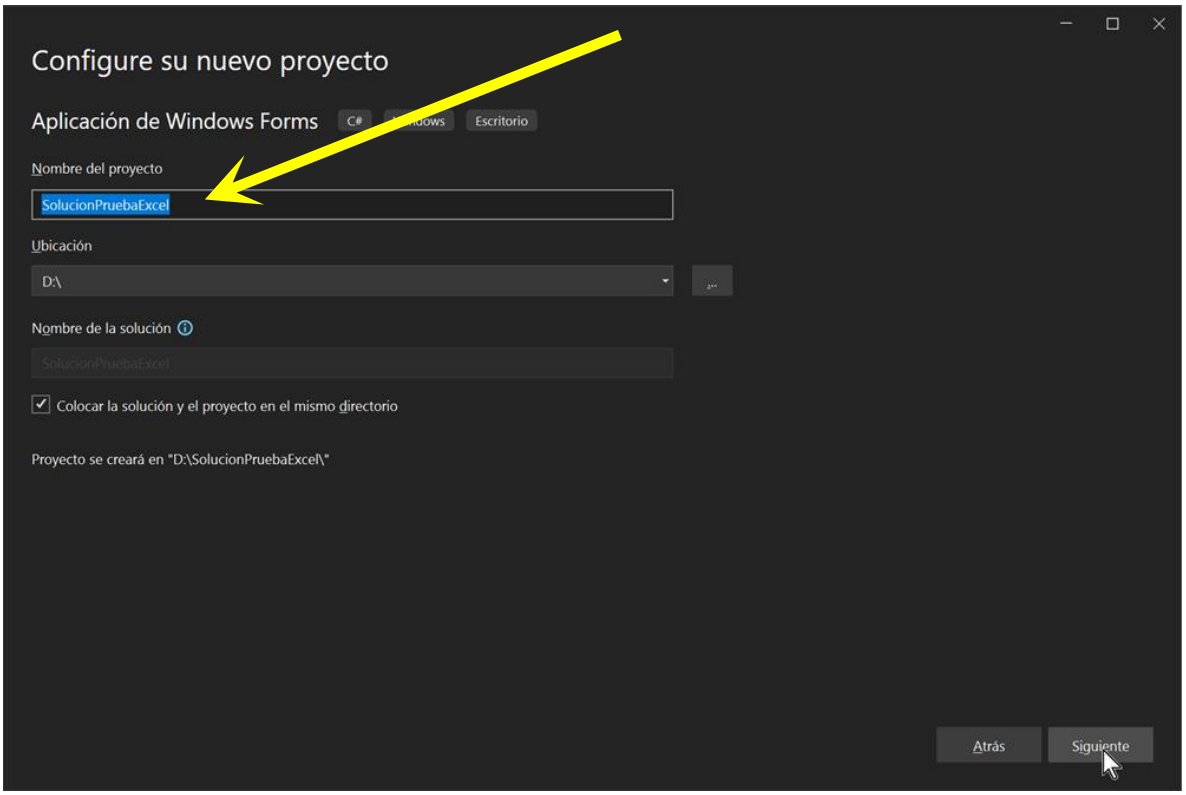
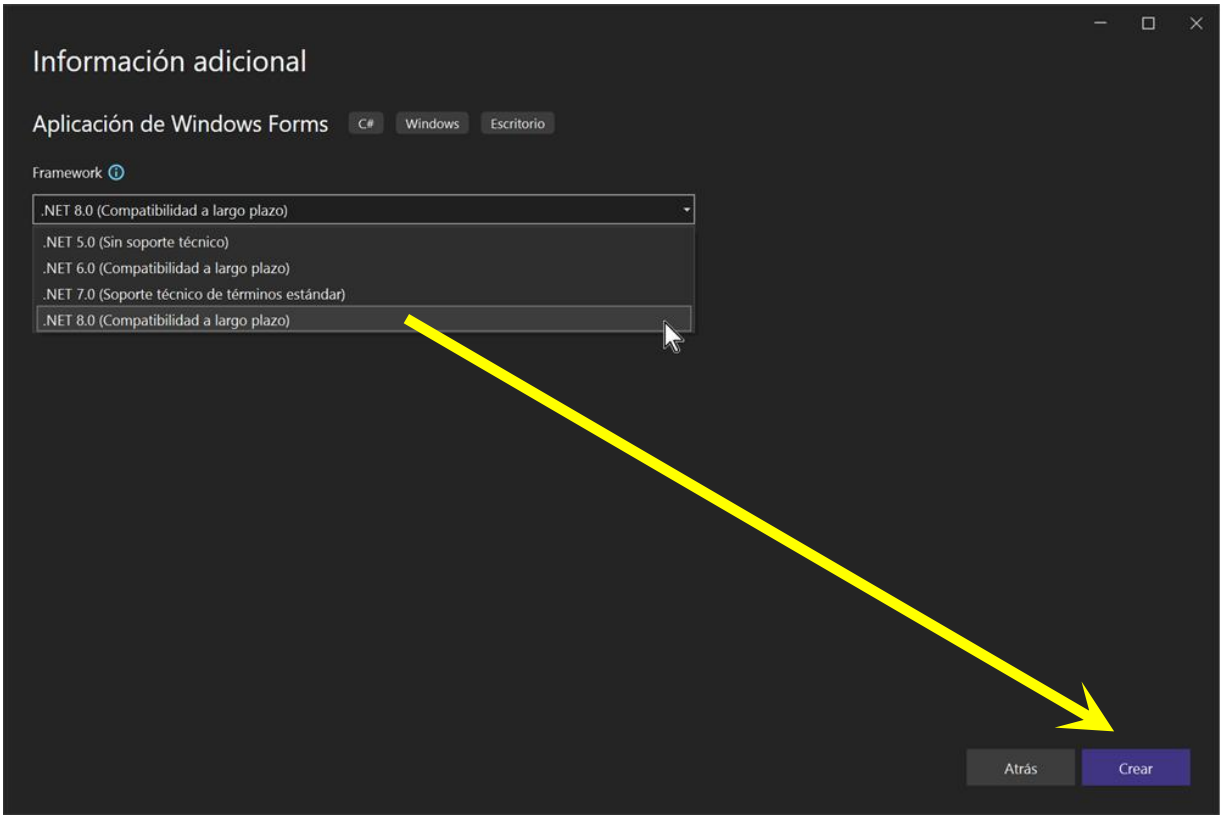
Aprendiendo a trabajar con Excel

En el presente documento se muestra paso a paso:
1) Seleccione exactamente la primera entrada (**NO LA SEGUNDA**)





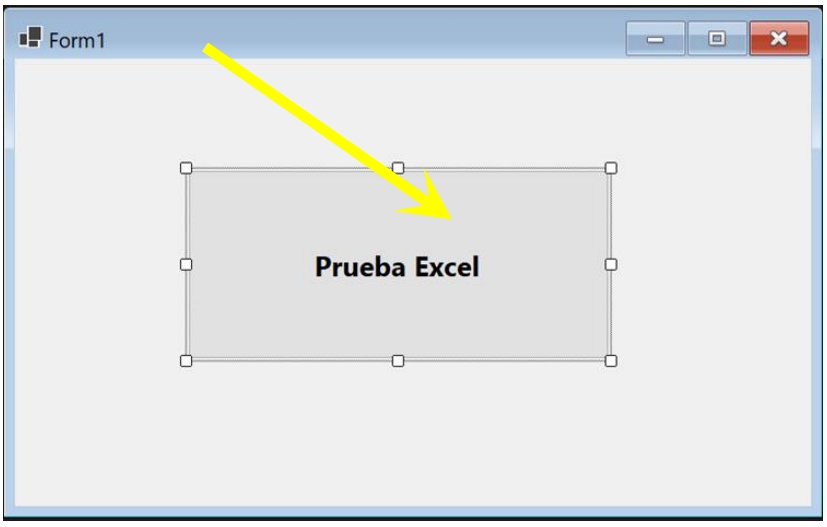
Ahora ponga en el nombre del proyecto **PruebaExcel** y de click en **Siguiente**:



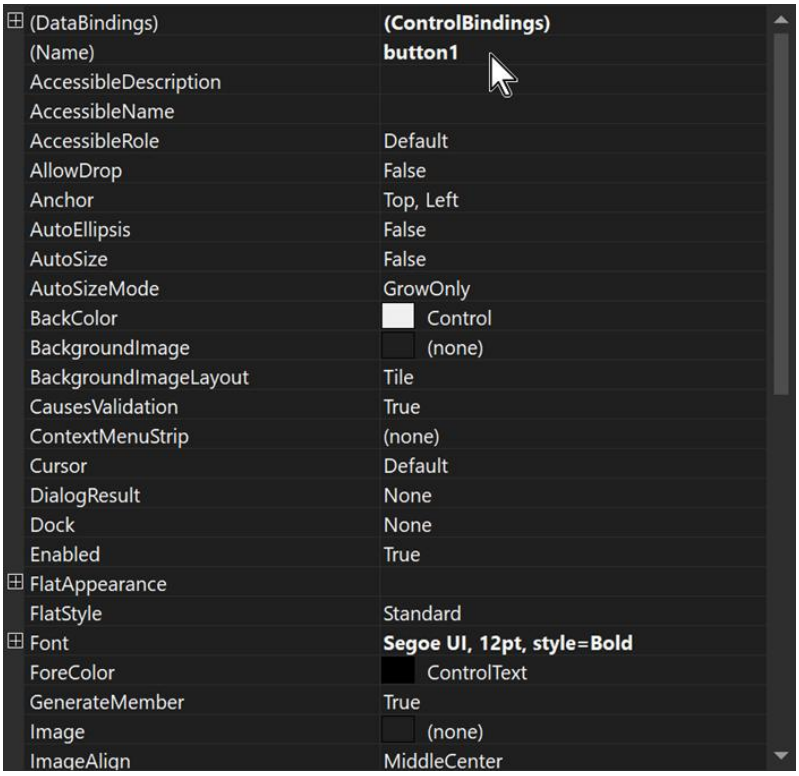
Escoja ahora el Framework con el que se va a trabajar,

ponga .NET 8.0 que es el que
Microsoft anuncia como de compatibilidad para futuras versiones independientemente de que presenta las últimas actualizaciones:

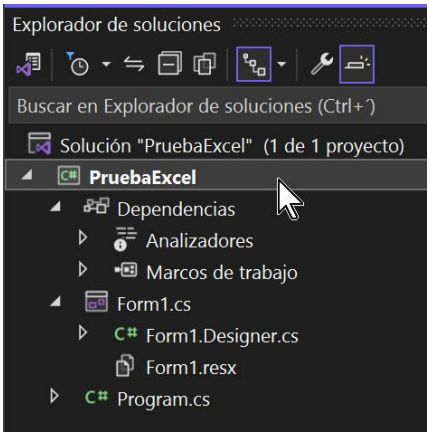
Inmediatamente de que se cree el nuevo proyecto asigne un botón (**button1)**, con el nombre **Prueba Excel** y dé doble click sobre el:



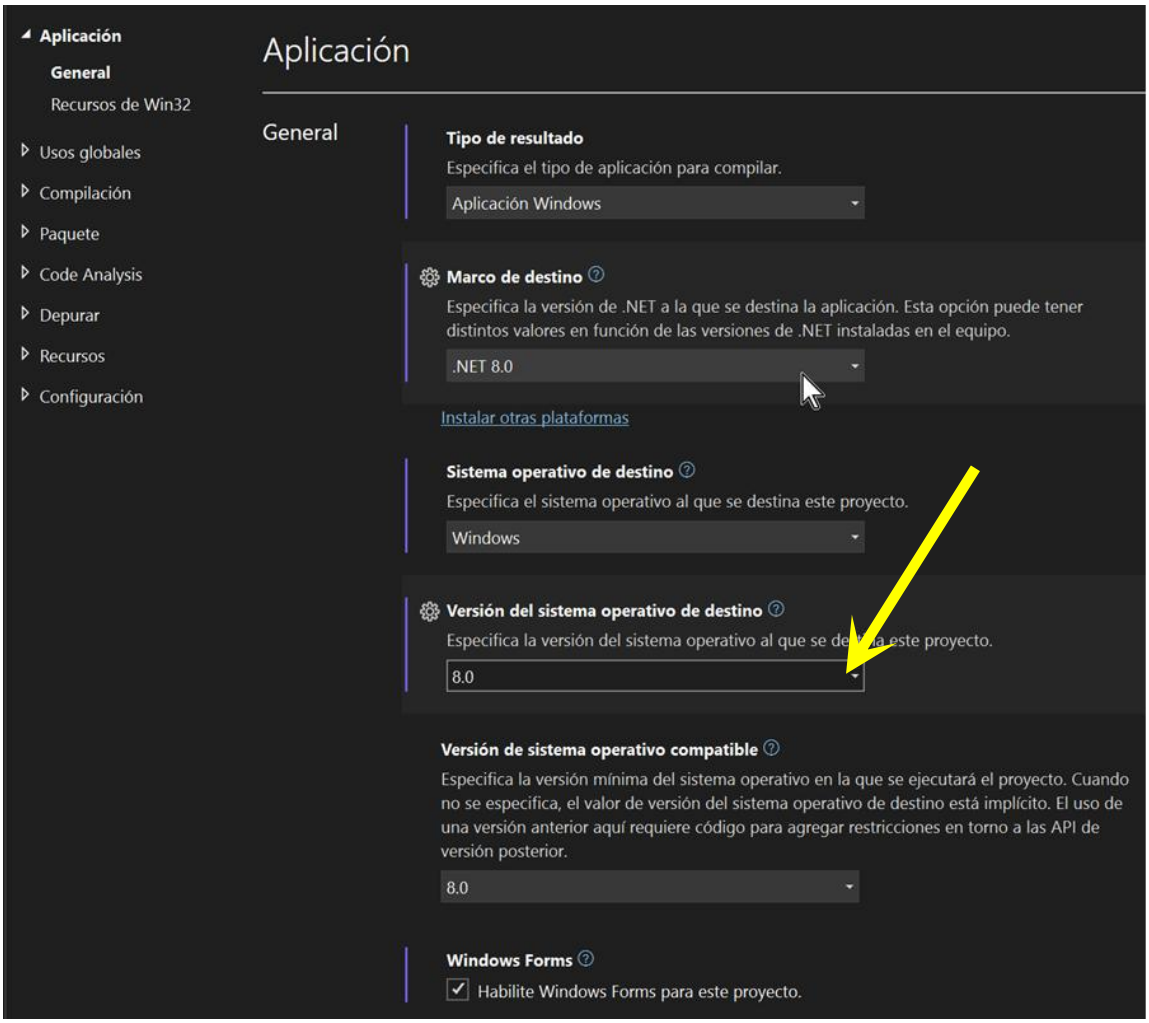
La ventana de propiedades aparece de la forma siguiente:



Verá que el explorador de soluciones aparece por primera vez de la forma siguiente:



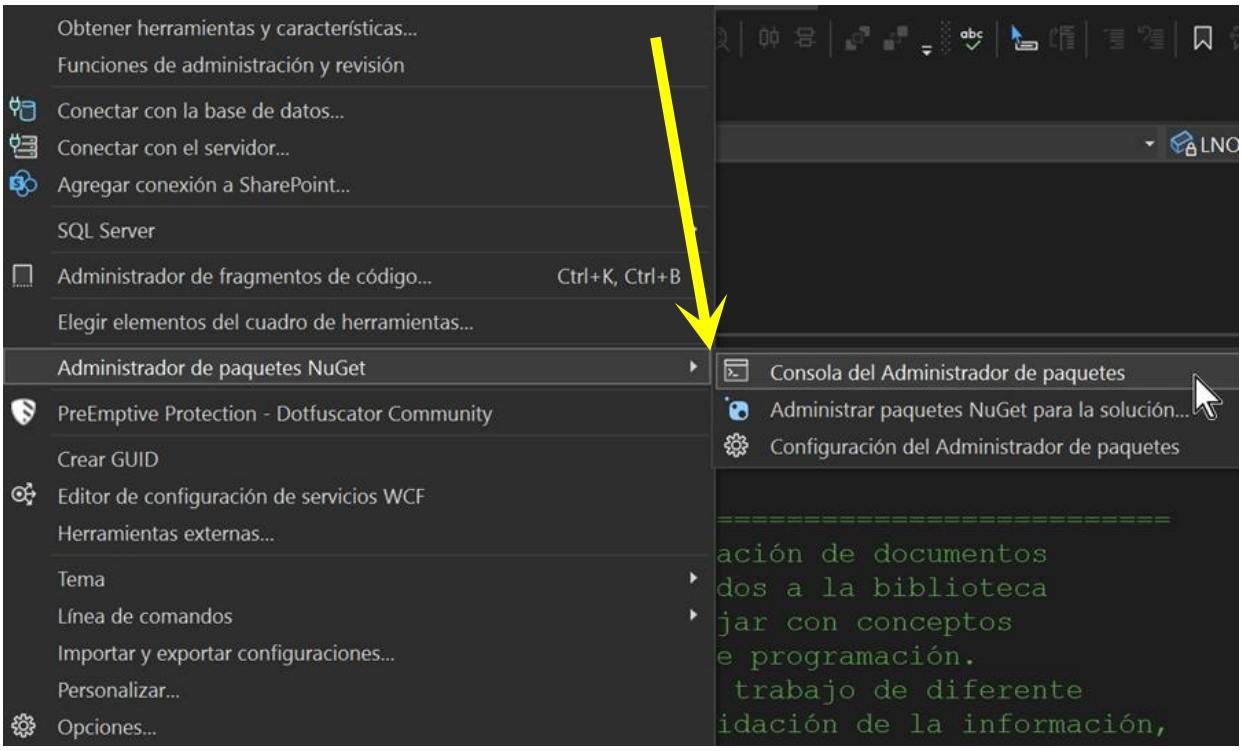
Ahora al dar click derecho en el nombre del proyecto y pedir sus propiedades, aparece en la ventana del centro, todas las propiedades de dicho proyecto según se muestra a continuación:



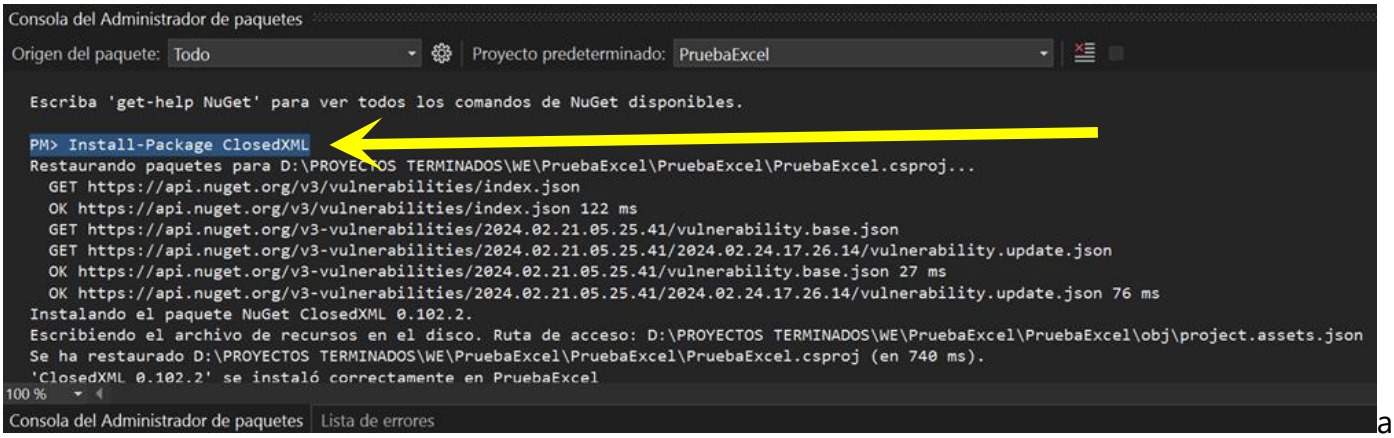
Seleccione la versión 8.0 en todos los ComboBox de la imagen superior. Esto va a asegurar de que estamos trabajando con .NET 8.0 en todos los niveles.

Pues bien, ahora comenzamos con la adaptación del proyecto para que pueda trabajar con documentos Excel, por tal razón comenzamos adicionando el **paquete Nuget (Package)**, siguiendo los siguientes pasos:

Damos click en Herramientas del menú superior de Visual Studio a fin de seleccionar el paquete de Excel con el que vamos a trabajar:

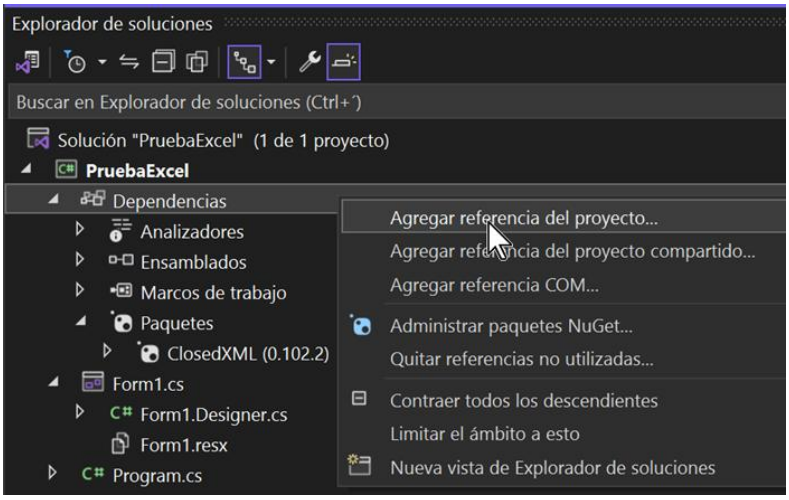


A continuación se adiciona el paquete denominado **ClosedXML** y damos **Enter**

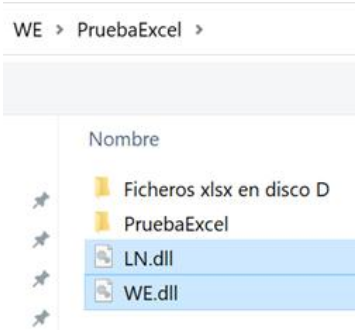


ap
areciendo a continuación los mensajes asociados a este proceso de adición del package de Excel.

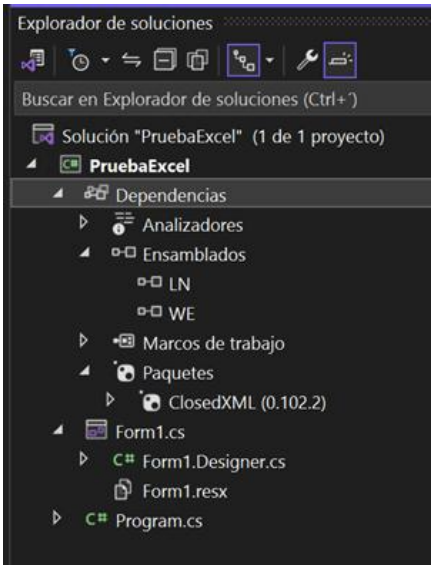
A partir de este momento se procede a adicionar las dos .DLL que usará el programa:



Dar al botón **Examinar** y escoger estas dos DLL, y la agregamos al proyecto:



Se debe observar las nuevas adiciones en el Explorador de Soluciones, por una parte las dos bibliotecas LN (**L**ibrary **N**ew) y WE (**W**ord & **E**xcel) dentro de los ensamblados, y por la otra el nuevo paquete CloseXML que contiene todas las componentes necesarias para el Excel.



==========

Ahora sustituya completamente todo el contenido de la página del código que ahora aparece

originalmente de esta forma:

```
namespace PruebaExcel
    {
    public partial class Form1 : Form
        {
        public Form1()
            {
            InitializeComponent();
            }

        private void button1_Click(object sender, EventArgs e)
            {

            }
        }
    }
```

==========

por el código del programa de ejemplo que adjuntamos a continuación:

==========

```
using ClosedXML.Excel;
using DocumentFormat.OpenXml.Vml;
using System;
using System.Windows.Forms;
using LN;
using WE;
namespace PruebaExcel
   {
//================================================================
// Autor: Lic. Hector Fortun (24/Enero/2024)
//================================================================
   // Programa de ejemplo de creación y modificación de documentos
   // EXCEL bajo el uso de los métodos adicionados a las bibliotecas
   // LN (Library New) creada para trabajar con conceptos que usualmente
   // consumen bastante tiempo de programación, al igual que con la biblioteca
   // WE (Word & Excel). Esta última comprende los métodos concebidos para
   // trabajar tanto con Word, como con Excel y sus equivalentes NO Microsoft,
   // como Ashampoo Office y otros más.
   // LN por otra parte, abarca varios perfiles de trabajo de diferente
   // tipo como son por ejemplo, métodos de validación de la información,
   // métodos para mensajería, trabajo con fecha y hora, llamadas a procedimientos
   // y objetos de Windows, Mensajes de Error o Informativos, etc.
   // Todo la anterior, siempre con el objetivo principal de hacer fácil lo difícil y ahorrar tiempo.
   // El objetivo principal ha sido el de crear una lógica simple y a la vez suficiente para facilitarle
   // al programador de C# bajo Visual Studio el trabajo de programación.
   //


   // Nota Importante:
   //
```

```
    // Dado que originalmente el poder programar para MSExcel requería
    // que estuviera instalado el MSOffice en su PC por usar a una librería
    // requerida por Microsoft (Interop), pues decidimos apoyarnos en
    // algunos Package de Nuget, como DocumentFormat.OpenXml (para Word) y
    // ClosedXML (para EXCEL), ambos con licencia MIT.
    // Dada mi experiencia personal con los anteriores observé que se
    // resistían un poco a romper la inercia y decidí estructurar a estas
    // herramientas de una forma muy simple que ayudara al programador a
    // avanzar con rapidez en la puesta a punto rápida de las tareas mas
    // cotidianas.
    // No obstante, dejando las puertas abiertas para que cuando el
    // programador lo decida pues utilice las herramientas originales del
    // fabricante, al cual le estamos muy agradecidos por tener la idea
    // de poder trabajar con equivalentes al Word y al Excel de Microsoft
    // sin necesidad de tenerlos instalados en nuestra PC.
    //
    // Es conveniente aclarar que ambos paquetes permiten el uso libre, también
    // en aplicaciones de índole comercial.

//=============================================================================
    public partial class Form1 : Form
        {
        //-------------------------------------------------------
        //              Zona de Variables Globales
        //-------------------------------------------------------
        LN.LN LNOBJ = new LN.LN();
        WE.WE OBJ = new WE.WE();
        //-------------------------------------------------------
        int        C = 0;
        int        F = 0;
        string      B2 = "";
        string      B3 = "";
        string      B4 = "";
        string      celda = "";
        float       B2Float = 0;
        float       B3Float = 0;
        float       B4Float = 0;
        float       B6Float = 0;
        string      TipoDato = "";
        XLDataType  XX = new XLDataType();
        //-------------------------------------------------------------
        public Form1()
            {
            InitializeComponent();
            }
```

```
private void button1_Click(object sender, System.EventArgs e)
   {
   try
      {
      OBJ.ExcelCreateDocExcel(@"D:\AAANuevo.xlsx", "Hoja1");
      // Abrir al fichero creado en el paso anterior para
      // trabajar con el.
      OBJ.ExcelFile  = OBJ.ExcelOpenFile(@"D:\AAANuevo.xlsx");
      // Abrir la hoja de cálculo (sheet) y asignarla a
      // ExcelSheet
      OBJ.ExcelSheet = OBJ.ExcelOpenSheet(1);

      OBJ.ExcelFormatTitles(1, "NOMBRE", "Segoe UI", 18, true, true,
                        XLColor.Red, XLColor.Yellow);
      OBJ.ExcelFormatTitles(2, "SALARIO", "Segoe UI", 18, true, true,
                        XLColor.Red, XLColor.Yellow);
      OBJ.ExcelFormatCell(2, 1, "Arial", "Arial", 18, true, false,
                        XLColor.DarkBlue, XLColor.Turquoise);
      OBJ.ExcelFormatCell(3, 1, "Times New Roman", "Times New Roman",
                        16, true, false,
                        XLColor.Black, XLColor.White);
      OBJ.ExcelFormatCell(4, 1, "Courrier New", "Courrier New",
                        14, true, false,
                        XLColor.Red, XLColor.Yellow);
      OBJ.ExcelFormatCell(2, 2, "250.50", "Times New Roman",
                        18, true, false,
                        XLColor.Turquoise, XLColor.DarkBlue);
      OBJ.ExcelFormatCell(3, 2, "300.48", "Arial", 16, true, false,
                        XLColor.White, XLColor.Black);
      OBJ.ExcelFormatCell(4, 2, "600.44", "Segoe UI", 14, true, false,
                        XLColor.Yellow, XLColor.Red);

      OBJ.ExcelSheet.Row(1).AdjustToContents();
      OBJ.ExcelSheet.Column(1).AdjustToContents();
      OBJ.ExcelSheet.Column(2).AdjustToContents();

      OBJ.ExcelSheet.Row(2).AdjustToContents();
      OBJ.ExcelSheet.Column(1).AdjustToContents();
      OBJ.ExcelSheet.Column(2).AdjustToContents();

      OBJ.ExcelSheet.Row(3).AdjustToContents();
      OBJ.ExcelSheet.Column(1).AdjustToContents();
      OBJ.ExcelSheet.Column(2).AdjustToContents();
```

```
OBJ.ExcelSheet.Row(4).AdjustToContents();
OBJ.ExcelSheet.Column(1).AdjustToContents();
OBJ.ExcelSheet.Column(2).AdjustToContents();

OBJ.ExcelSheet.Row(5).AdjustToContents();
OBJ.ExcelSheet.Column(1).AdjustToContents();
OBJ.ExcelSheet.Column(2).AdjustToContents();

OBJ.ExcelSheet.Row(6).AdjustToContents();
OBJ.ExcelSheet.Column(1).AdjustToContents();
OBJ.ExcelSheet.Column(2).AdjustToContents();

//=======OPERACIONES ARITMETICAS CON CELDAS EN UNA COLUMNA=======
B2 = OBJ.ExcelSheet.Cell("B2").Value.ToString();
B3 = OBJ.ExcelSheet.Cell("B3").Value.ToString();
B4 = OBJ.ExcelSheet.Cell("B4").Value.ToString();
//-----------------------------------------------------------------
B2Float = float.Parse(B2);
B3Float = float.Parse(B3);
B4Float = float.Parse(B4);
//-----------------------------------------------------------------
B6Float = B2Float + B3Float + B4Float;
//-----------------------------------------------------------------
OBJ.ExcelSheet.Cell("B6").Value = B6Float.ToString();

//------------FORMATEAR LA COLUMNA DEL RESULTADO--------------------
OBJ.ExcelFormatCell(6, 2, OBJ.ExcelSheet.Cell("B6").Value.ToString(),
                    "Arial", 18, true, false,
                    XLColor.DarkRed, XLColor.Yellow);
//-----------------------------------------------------------------
// El resultado de esta operacion aritmética "(B4 + B6 + A7) / 3"
// queda en A8
//-----------------------------------------------------------------
// Es conveniente observar que la celda B5 no tiene contenido,
// sin embargo se acepta en la operación como si el valor fuera cero.
//-----------------------------------------------------------------
OBJ.ExcelFormula("A8", "(B4 + B5 + B6 + A7) / 3");



//======ASIGNACION DE UN TIPO DE DATOS A UNA CELDA=====
OBJ.ExcelSheet.Cell(7,1).Value = "123.45";
B2 = OBJ.ExcelSheet.Cell(7,1).DataType.ToString();
XX = OBJ.ExcelSheet.Cell(7,1).DataType;
OBJ.ExcelFormatCell(7, 1, OBJ.ExcelSheet.Cell(7,1).Value.ToString(),
```

```
                    "Arial", 14, true, false,
                    XLColor.DarkRed, XLColor.Yellow);
   celda = OBJ.ExcelGetCell("A7");
   XX = XLDataType.Number;

   if (XX == XLDataType.Text)
      {
      OBJ.MensINFO("Cell A7 = Text");
      }
   if (XX == XLDataType.Number)
      {
      OBJ.MensINFO("Cell A7 = Number");
      }
   //-----------------------------------------------------------------
   OBJ.ExcelWriteCellRC(5, 1, "AAA");
   OBJ.ExcelWriteCellRC(6, 1, "BBB");

   OBJ.ExcelWriteCell("B3", "CCC");
   OBJ.ExcelWriteCell("B2", "DDD");
   // Obtiene el valor de la celda (3,2)
   celda = OBJ.ExcelGetCellRC(3, 2);
   //Obtiene el tipo de Dato de (4,2)
   TipoDato = OBJ.ExcelGetCellDataTypeRC(4, 2);
   //Obtiene el tipo de Dato de "B2"
   celda = OBJ.ExcelGetCellDataType("B2");

   OBJ.MensINFO("# Columnas: " + C.ToString() + "\n" +
            "# Filas: " + F.ToString() +
            "\nCelda(3,2) = " + celda +
            "\nTipo de Dato (4,2) = " + TipoDato +
            "\nTipo de Dato (2,2) = " + celda);

   OBJ.MensINFO("Fin del Proceso");
   }
catch
   {
   OBJ.MensINFO("Excepción Detectada - Se cancela el programa\n" +
            e.ToString());
   Application.Exit();
   }
OBJ.ExcelSaveFile(@"D:\AAAOutput.xlsx");
LNOBJ.MensINFO("Prueba de Excel Finalizada Correctamente" +
                  @"\nFichero Resultante en D:\AAAOutput.xlsx");
Application.Exit();
}
```

```
    }
}
```

Y ya esta terminado el proceso de configuración para ejecutar al programa de ejemplo de Excel. Compile y ejecute al mismo y verá después de la ejecución que en la raiz del disco D:\ aparecen estos dos nuevos ficheros **GENERADOS POR EL PROGRAMA**:

